

CẨM NANG TOÀN TẬP

Benchmark mật mã hậu lượng tử trên lưới (ML-KEM / ML-DSA vs RSA / ECC)

Hướng dẫn từng-lệnh-một cho người mới bắt đầu, tách riêng hai thiết bị
x86_64 (PC/laptop) và ARM (Raspberry Pi 4)

NT219 — Cryptography Capstone

June 12, 2026

Abstract

Tài liệu này dạy lại *toàn bộ* repo từ con số không: mỗi câu lệnh đều có “Làm gì / Tại sao / Nguồn / Kiểm ngay”; mỗi công nghệ đều có lý do chọn và đường link tài liệu gốc; mỗi giả thuyết nghiên cứu đều được nối thẳng tới phép đo trả lời nó. Cấu trúc theo đúng trực thiết bị: **Phần I** đi trọn vẹn trên x86_64, **Phần II** đi trọn vẹn trên ARM, **Phần III** là khâu tổng hợp chung, kèm phụ lục tra cứu 51 file của repo và FAQ “tại sao” cho người mới.

Contents

PHẦN MỞ ĐẦU — Đồ án này là gì, và bản đồ tổng	2
1 PHẦN I — THIẾT BỊ 1: PC x86_64 (Ubuntu 22.04/24.04)	4
1.1 Lý thuyết nền cho người mới — đọc 10 phút trước khi chạm máy	4
1.2 Bước 0 — Lấy repo về máy	4
1.3 Bước 1 — install_prereqs.sh: cài đúng và đủ gói	5
1.4 Bước 2 — versions.env & setenv.sh: ghim phiên bản và bật môi trường	5
1.5 Bước 3 — build_openssl.sh: dựng nền móng (OpenSSL 3.6.2 PQC-native)	5
1.6 Bước 4 — verify_env.sh: cánh cổng và tờ biên bản	6
1.7 Bước 5 — make: dựng harness đo bench_evp	6
1.8 Bước 6 — build_liboqs.sh: dựng HAI cây cho thí nghiệm RQ2	7
1.9 Bước 7 — fetch_pqclean.sh: nguồn duy nhất cho code-size từng thuật toán	7
1.10 Bước 8 — build_oqsprovider.sh: đường tích hợp kế nhiệm	7
1.11 Bước 9 — gen_tls_certs.sh: chứng thư cho trực chữ ký	8
1.12 Bước 10 — ĐO WP2: microbenchmark K-batch (số liệu xương sống)	8
1.13 Bước 11 — ĐO WP5: bộ nhớ và kích thước mã	8
1.14 Bước 12 — ĐO WP3: đổ số ref/opt thành CSV	9
1.15 Bước 13 — ĐO WP4: TLS handshake, BA phương án cùng một thước đo	9
1.16 Bước 14 — make analyze: từ CSV thành bảng và biểu đồ	12
1.17 Bước 15 — Docker (tái lập, tùy chọn trên x86)	12
1.18 Bước 16 — Tùy chọn x86 (mở rộng chiều dữ liệu) + nghiệm thu	12
2 PHẦN II — THIẾT BỊ 2: ARM Raspberry Pi 4 (aarch64, Ubuntu)	13
2.1 ARM khác x86 ở đâu — lý thuyết bắt buộc trước khi thuê máy	13

2.2 Pipeline ARM — toàn bộ lệnh, theo đúng thứ tự	13
2.3 Cross-compile từ x86 — bảng chứng năng lực (KHÔNG phải nguồn số)	15
2.4 Tùy chọn ARM	15
3 PHẦN III — Sau khi có số ở cả hai máy: tổng hợp, báo cáo, demo	16
A Phụ lục A — Bản đồ 51 file: mỗi file một dòng công dụng	16
B Phụ lục B — Toàn bộ biến môi trường (không hardcode)	17
C Phụ lục C — FAQ “tại sao” cho người mới (12 câu hay gặp nhất)	17

PHẦN MỞ ĐẦU — Đề án này là gì, và bản đồ tổng

0.1 Ba câu hỏi nghiên cứu → ba hướng đi → công nghệ tương ứng

Đề bài đặt ba câu hỏi (RQ) và một giả thuyết. Mọi thứ trong repo tồn tại để trả lời chúng — nếu một file không phục vụ RQ nào, nó không có mặt.

Câu hỏi / giả thuyết	Hướng đi (phép đo)	Công nghệ — vì sao chọn
RQ1: PQC đắt hơn RSA/ECDSA bao nhiêu (tính toán, băng thông) trên x86 và ARM?	Microbenchmark cùng một mặt cắt đo cho mọi thuật toán; đo kích thước khóa/chữ ký; đo trên <i>hai</i> máy thật.	bench_evpcpp qua EVP API của OpenSSL 3.6.2 — một API chung cho cả RSA/ECC lẫn ML-KEM/ML-DSA nên so sánh công bằng; đồng hồ CLOCK_MONOTONIC_RAW đúng nguyên văn đề §8.3.
RQ2: Tối ưu ARM (NEON, cờ biên dịch) có kéo PQC về mức khả thi trên SBC?	Thí nghiệm ghép cặp: <i>cùng</i> liboqs, <i>cùng</i> máy, hai bản build chỉ khác đúng một biến (SIMD tắt/bật).	liboqs hai cây ref/opt (build_liboqs.sh); chạy speed_kem/speed_sig của chính liboqs — đúng phương pháp OQS từng dùng trong hạ tầng profiling của họ.
RQ3: Handshake hybrid (ECDHE + ML-KEM) có overhead chấp nhận được?	Đo TLS 1.3 handshake thật: latency + throughput, trên ma trận [chứng thư × nhóm trao đổi khóa], cả loopback lẫn LAN.	Ba server cùng lỗi libssl: s_server (cô lập mật mã), nginx build từ nguồn (server sản xuất, trực đơn/đa luồng), tls_mini_server.c tự viết (đo phía server + bằng chứng group).
Giả thuyết của đề: PQC tốn băng thông/khóa lớn hơn nhưng tối ưu phần mềm đưa hiệu năng về đủ dùng.	Nếu đúng: cột kích thước sẽ thua rõ, cột tốc độ sau tối ưu sẽ ngang/hơn RSA.	Chính bảng analysis_out/tables.md do analyze.py sinh ra là nơi giả thuyết được đối chiếu với số.

0.2 Bản đồ repo — nhìn một phát biết mọi thứ ở đâu

```
repo/  
  Makefile <- cong tac tong: make / bench / memory / codesize / tls / analyze / tlsmini /  
    tlsclient  
  README.md <- cua chinh: quickstart 2 may + bang chon phuong an + bang bien env  
  scripts/ <- 21 script: build moi truong + do dac + phan tich  
  src/ <- 3 nguon tu viet: bench_evpcpp, tls_mini_server.c, tls_timer_client.c  
  cmake/ <- toolchain cross-compile aarch64  
  docker/ <- Dockerfile x86_64 (tai lap)  
  data/ <- CSV ket qua (cam ket vao git!) + data/raw/<arch>/ bang chung tho  
  analysis_out/ <- bang + bieu do (cung cam ket: la deliverable)  
  docs/ <- kích ban do, ho so thiet ke, bao cao LaTeX, cam nang nay
```

Tại sao: Bố cục này ánh xạ mẫu §17 của đề (bảng ánh xạ chi tiết nằm cuối README): benchmarks/micro của đề chính là cặp src/bench_evpcpp+scripts/run_micro.sh; benchmarks/tls là cụm script TLS; tools/parsers là analyze.py. Thư mục energy/ của đề *cố ý không có* — đề §13 cho phép khi thiếu thiết bị đo điện (Monsoon/INA219), và quyết định đó có hồ sơ ở mục Limitations của báo cáo.

0.3 Bốn định luật xuyên suốt — đọc trước khi gõ bất kỳ lệnh nào

1. **Định luật người thuê nhà:** mọi phần mềm TLS (nginx, s_server, server tự viết, curl, wrk) đều “thuê” mật mã từ thư viện OpenSSL nó link. Muốn có ML-KEM/ML-DSA thì *thư viện* phải là 3.5+ — đổi config, đổi cờ đều vô ích nếu thư viện bên dưới là 3.0. Đã chứng minh bằng mã nguồn: nginx gọi `SSL_CTX_set1_curves_list` (`ngx_event_openssl.c:1799`), server tự viết gọi `SSL_CTX_set1_groups_list` — hai tên của *cùng một hàm* libssl.
2. **Định luật bốn con số:** chỉ so trực tiếp hai số đo trên *cùng một máy*. Muốn nói chuyện giữa hai máy thì so *tỉ lệ* (`opt/ref` của máy này vs `opt/ref` của máy kia). Vì vậy mọi kịch bản đo chạy đủ trên từng máy.
3. **Định luật tham số:** mặc định trong code chỉ là fallback; mọi vòng lặp/cổng/thời lượng chỉnh được qua biến môi trường (bảng đầy đủ ở Phụ lục B). `BENCH_ITERS=5000` `make bench` và `make bench BENCH_ITERS=5000` đều tới đích (đã test sống).
4. **Định luật bằng chứng:** số chưa push lên git coi như chưa tồn tại (máy thuê có thể bị thu hồi); mọi cơ chế trong repo đều đã chạy thật ít nhất một lần — nhật ký 10 mục trong `docs/THIET_KE_CHI_TIET.md` mục 9.

1 PHẦN I — THIẾT BỊ 1: PC x86_64 (Ubuntu 22.04/24.04)

1.1 Lý thuyết nền cho người mới — đọc 10 phút trước khi chạm máy

Lý thuyết — PQC là gì và vì sao phải đo. Máy tính lượng tử đủ lớn sẽ phá RSA và ECC (thuật toán Shor). NIST đã chuẩn hóa thể hệ thay thế dựa trên *lưới* (lattice): **ML-KEM** (FIPS 203, tên cũ Kyber) cho trao đổi khóa, **ML-DSA** (FIPS 204, tên cũ Dilithium) cho chữ ký. Giá phải trả: khóa/chữ ký/bản mã *to hơn nhiều* và chi phí tính toán *khác* — to hơn ở đâu, nhanh chậm chỗ nào trên phần cứng thật, chính là việc đo án đo. Nguồn: FIPS 203 <https://csrc.nist.gov/pubs/fips/203/final>, FIPS 204 <https://csrc.nist.gov/pubs/fips/204/final>.

Lý thuyết — KEM khác chữ ký — và hệ quả lên TLS. Một handshake TLS có *hai* việc mật mã độc lập: (1) **trao đổi khóa** (KEM/DH) — tạo bí mật chung tạm thời, dùng một lần rồi bỏ; (2) **xác thực danh tính** — server ký để chứng minh nó là chủ certificate. ML-KEM thay việc (1); ML-DSA thay chữ ký trong việc (2). KEM **không bao giờ nằm trong certificate** (xác nhận bởi maintainer OpenSSL V. Dukhovni trên openssl-users) — vì vậy ma trận đo WP4 có *hai trục độc lập*: trục group (KEM) và trục chứng thư (chữ ký).

Lý thuyết — Mức an toàn tương đương — để so cho công bằng. So ML-KEM-768 với RSA-2048 mà kết luận “PQC nhanh hơn” là so lê với táo. Bảng quy đổi (FIPS 203/204 + NIST SP 800-57 <https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final>): mức 1 ($\approx$ AES-128): ML-KEM-512/ML-DSA-44 $\leftrightarrow$ RSA-3072 $\leftrightarrow$ P-256; mức 3: ML-KEM-768/ML-DSA-65 $\leftrightarrow$ RSA-7680 $\leftrightarrow$ P-384; mức 5: ML-KEM-1024/ML-DSA-87 $\leftrightarrow$ RSA-15360 $\leftrightarrow$ P-521. RSA-2048 ($\approx$ 112-bit) vẫn được đo vì là baseline thực dụng phổ biến — nhưng khi viết kết luận phải chú thích nó dưới mức 1.

Lý thuyết — Benchmark nghĩa là đo gì. Bốn họ chỉ số của đề §9: **latency** (một phép toán mất bao lâu — báo median/mean/p95/CI chứ không chỉ trung bình, vì máy tính có nhiều: median “người đứng giữa hàng” miễn nhiễm với vài vòng đột biến); **throughput** (mỗi giây làm được bao nhiêu việc dưới tải song song); **memory** (đỉnh RAM tiến trình chiếm — peak RSS); **code size** (thư viện nặng bao nhiêu byte trên đĩa — text/data/bss). Hai khái niệm hay nhầm: memory đo lúc *chạy*, code size đo lúc *nằm im*.

Lý thuyết — Warm-up và K-batch — vì sao không đo một phát ăn ngay. Vòng đầu tiên luôn chậm bất thường (cache lạnh, thư viện nạp lười) — bằng chứng sống trong repo: server tự viết ghi kết nối #1 mất 7996 μ s trong khi #2-3 chỉ $\sim$ 3800 μ s. Vì vậy: loại BENCH_WARMUP vòng đầu, và chạy K=5 *batch* độc lập rồi lấy median-of-medians (đề §7.5) — một batch xui (nhiệt độ, tiến trình nền) không phá được kết quả.

1.2 Bước 0 — Lấy repo về máy

```
git clone <URL-repo> && cd <ten-repo>
chmod +x scripts/*.sh
```

Làm gì: Tải mã nguồn đồ án và cấp quyền thực thi cho 21 script.

Tại sao: chmod +x cần thiết vì quyền thực thi có thể mất khi nén/chuyển máy; thiếu nó lệnh bash scripts/... vẫn chạy nhưng ./scripts/... sẽ báo Permission denied — chuẩn hóa ngay từ đầu đỡ một câu hỏi về sau. Nếu file từng mở trên Windows và lỗi '\$\'\'\'r': command not found: sed -i 's/\\r\\\$/' scripts/*.sh (xóa ký tự xuống dòng kiểu Windows).

1.3 Bước 1 — install_prereqs.sh: cài đúng và đủ gói

```
| bash scripts/install_prereqs.sh
```

Làm gì: Cài ba đợt gói: (1) toolchain biên dịch — build-essential, cmake, ninja-build, perl, git; (2) công cụ đo — GNU time, linux-tools (perf), apache2-utils (ab), wrk, tshark, htop, cpupower; (3) phân tích — python3-matplotlib (pandas tùy chọn).

Tại sao: Mỗi đợt ánh xạ một mục đích: §8.2 đòi đúng bộ toolchain đó (OpenSSL cần perl để Configure; liboqs khuyên ninja); §7.4 gọi đích danh /usr/bin/time -v và perf stat; §16 gọi matplotlib. **Cố ý KHÔNG cài libssl-dev:** trộn header OpenSSL hệ thống với bản tự build là đúng lớp bug nguy hiểm nhất của dự án (mọi build đều trở tường mình vào ~/pqc/openssl).

Nguồn: danh sách gói đối chiếu kho Ubuntu 24.04; perf cần khớp kernel đang chạy nên script cài thêm linux-tools-\$(uname -r) — kernel cloud/Pi mang hậu tố riêng (-aws/-raspi).

Kiểm ngay: hộp thoại tshark hiện ra → chọn **Yes**, sau đó sudo usermod -aG wireshark \$USER và đăng nhập lại (quyền nhóm chỉ nạp lúc login).

1.4 Bước 2 — versions.env & setenv.sh: ghim phiên bản và bật môi trường

Lý thuyết — Vì sao ghim tag. “Reproducible” (để §7.3) nghĩa là hai máy build hai ngày khác nhau vẫn ra cùng một phần mềm. Branch trôi theo thời gian; tag thì bất biến. Toàn bộ phiên bản nằm một chỗ: OPENSSL_TAG=openssl-3.6.2, LIBOQS_TAG=0.14.0, OQS_PROVIDER_TAG=0.10.0, NGINX_TAG=release-1.30.2. Muốn đổi: sửa một dòng, xóa nguồn cũ trong ~/pqc/src, chạy lại với FORCE=1.

```
| source scripts/setenv.sh # MỌI terminal mọi đeu phai chay
| which openssl # phai tra ve ~/pqc/openssl/bin/openssl
```

Làm gì: Đặt PATH, LD_LIBRARY_PATH, PKG_CONFIG_PATH trở về OpenSSL của ta.

Tại sao: Phải source chứ không ./: biến môi trường đặt trong tiến trình con sẽ chết theo nó — ./setenv.sh chạy xong là biến bay màu. Repo chọn **không nường rpath** vào binary (đúng quy ước demos chính chủ OpenSSL <https://github.com/openssl/openssl/tree/master/demos>): thư viện được tìm lúc chạy qua LD_LIBRARY_PATH — binary di dời được, cơ chế tường mình. Bằng chứng sống về hậu quả khi quên: chạy ldd lên nginx thiếu biến này → loader té về libssl hệ thống và vắng OPENSSL_3.2.0 not found — binary không nạp nổi.

1.5 Bước 3 — build_openssl.sh: dựng nền móng (OpenSSL 3.6.2 PQC-native)

Lý thuyết — Vì sao phải tự build OpenSSL. Ubuntu 24.04 ship OpenSSL 3.0.x — không có ML-KEM/ML-DSA (hai họ này “added in OpenSSL 3.5” — nguyên văn tài liệu https://docs.openssl.org/3.5/man7/EVP_PKEY-ML-DSA/). Không có thư viện 3.5+ thì mọi phép đo PQC bất khả. Ta build 3.6.2 vào ~/pqc/openssl, không dựng OpenSSL hệ thống — hệ điều hành vẫn an toàn, gỡ sạch = rm -rf ~/pqc.

```
| bash scripts/build_openssl.sh # lan dau: chay du test suite (~20-40 phut)
| # cac lan sau / may yeu: SKIP_TESTS=1 bash scripts/build_openssl.sh
```

Làm gì: Bên trong script làm 6 việc, mỗi việc một lý do:

1. git clone --depth 1 --branch \$OPENSSL_TAG https://github.com/openssl/openssl.git → tải đúng bản ghim từ repo chính chủ; --depth 1 = nông, chỉ lấy snapshot (nhANH, NHẹ).
2. ./Configure --prefix=\$HOME/pqc/openssl → khai nơi cài; mọi thứ gom dưới ~/pqc.
3. make -j \$JOBS → biên dịch song song theo số nhân CPU (đọc từ versions.env).

4. `make test` (bỏ qua được bằng `SKIP_TESTS=1`) → bộ test chính chủ chứng minh bản build *đúng đắn* — chạy đủ ít nhất một lần mỗi máy làm bằng chúng; các lần rebuild sau được phép bỏ (quy ước giống CI của oqs-demos).
5. `make install_sw install_ssldirs` → `install_sw` cài gọn (bỏ man pages); `install_ssldirs` tạo `openssl.cnf` — **thiếu nó là lệnh `openssl req` chết**. Đây không phải lý thuyết: bản build đi tắt trong sandbox đã bỏ bước này và `req` fail thật với lỗi “Can’t open ../openssl.cnf”.
6. ghi docs/openssl.commit → bằng chứng tái lập: commit hash chính xác của bản đã build.

Kiểm ngay: `source scripts/setenv.sh && openssl version` → phải in OpenSSL 3.6.2; `openssl list -kem-algorithms | grep ML-KEM` → phải thấy ML-KEM-512/768/1024.

1.6 Bước 4 — `verify_env.sh`: cánh cổng và tờ biên bản

```
| bash scripts/verify_env.sh # phải in PASS
| cat docs/env_report_x86_64.txt # biên bản mọi trường của MAY NAY
```

Làm gì: Một script hai vai: (1) *cổng chặn* — kiểm OpenSSL đang dùng có ML-KEM/ML-DSA không, sai thì exit 1 kèm hướng dẫn sửa; (2) *biên bản* — ghi compiler, cờ build, CPU, governor ra docs/env_report_<arch>.txt.

Tại sao: Vai (1) chống lớp lỗi tệ nhất của mật mã: *fail im lặng* — âm thầm đo trên OpenSSL 3.0 hệ thống rồi ra số “có vẻ đúng”. Vai (2) trả lời nguyên văn đề §7.3: “Document compiler versions (gcc/clang), flags, and CPU governor settings”. Governor đọc từ `/sys/devices/system/cpu/cpu0/cpufreq/scaling_governor` (tài liệu kernel: <https://www.kernel.org/doc/html/latest/admin-guide/pm/cpufreq.html>).

Kiểm ngay: cả hai nhánh đã chạy thật: trở vào OpenSSL 3.0 → FAIL đúng; môi trường chuẩn → PASS + sinh report.

1.7 Bước 5 — `make`: dựng harness đo `bench_ev`

```
| make # biên dịch src/bench_ev.cpp -> build/bench_ev
```

Làm gì: Makefile đọc `versions.env` để biết OpenSSL ở đâu, tự dò `lib/` hay `lib64/` (OpenSSL chọn theo nền tảng), biên dịch `bench_ev.cpp` với `-O2 -Wall` và link `-lcrypto -pthread`.

Tại sao: Từng lựa chọn trong Makefile có lý do: `-O2` là mức tối ưu chuẩn đo lường (mã ta *bao quanh* phép đo cần nhanh nhưng không được đổi ngữ nghĩa như `-O3` đôi khi gây); `$(error ...)` khi thiếu OpenSSL — *fail to be loud*, thà chết sớm với thông điệp rõ còn hơn lỗi khó hiểu ở bước link; `-pthread` thay `-lpthread` là cách hiện đại (đặt cả cờ compile lẫn link).

Lý thuyết — EVP API là gì và vì sao chọn nó. EVP là “mặt tiền” thống nhất của OpenSSL: cùng một bộ hàm `EVP_PKEY_keygen/encapsulate/sign/verify` cho mọi thuật toán — RSA, ECDSA, ML-KEM, ML-DSA chỉ khác cái tên truyền vào. Đo qua EVP nghĩa là mọi thuật toán đi qua *cùng một lớp chi phí giao tiếp* → chênh lệch đo được là chênh lệch *thuật toán*, không phải chênh lệch cách gọi. Tên thuật toán lấy nguyên văn tài liệu: ML-KEM-768 (https://docs.openssl.org/3.5/man7/EVP_PKEY-ML-KEM/), ML-DSA-65 (https://docs.openssl.org/3.5/man7/EVP_PKEY-ML-DSA/).

Lý thuyết — Đồng hồ `CLOCK_MONOTONIC_RAW`. Đề §8.3 chỉ định đích danh. “Monotonic” = chỉ tiến không lùi (không bị NTP chỉnh giờ kéo ngược); “RAW” = không bị kernel nắn tần số. Đo hiệu năng mà dùng đồng hồ tường (`CLOCK_REALTIME`) là tự rước nhiễu. `bench_ev` còn đọc kèm bộ đếm chu kỳ CPU (`rdtsc/cntvct`) để *kiểm chéo*: tỉ lệ thời gian giữa hai thuật toán phải khớp tỉ lệ chu kỳ — và trên số thật đã khớp.

Kiểm ngay: ./build/bench_evpc mlkem 768 chạy thử một thuật toán; cú pháp <h> <tham số>: rsa 2048|3072|4096|15360, ecDSA|ecdh p256|p384|p521, mlkem 512|768|1024, mldsa 44|65|87.

1.8 Bước 6 — build_liboqs.sh: dựng HAI cây cho thí nghiệm RQ2

```
bash scripts/build_liboqs.sh ref # cây C thuần: SIMD TAT tuong minh
bash scripts/build_liboqs.sh opt # cây tối ưu: -march=native, AVX2 bat
```

Làm gì: Build liboqs 0.14.0 hai lần vào hai prefix riêng (~/pqc/liboqs-ref, ~/pqc/liboqs-neon), giữ lại thư mục build-*/tests chứa speed_kem/speed_sig.

Tại sao: Đây là *thí nghiệm ghép cặp* (đề §7.5 “paired comparisons”): cùng phiên bản, cùng máy, cùng compiler — biến độc lập *duy nhất* là tối ưu vi kiến trúc. ref ép OQS_OPT_TARGET=generic (tắt hẳn AVX2/NEON); opt để -march=native tự dò. Bản opt đặt OQS_DIST_BUILD=OFF — tài liệu cấu hình liboqs (<https://github.com/open-quantum-safe/liboqs/blob/main/CONFIGURE.md>) ghi rõ DIST_BUILD bật là build “chạy mọi nơi” (chọn SIMD lúc chạy); ta đo trên đúng máy này nên tắt để khai thác tối đa. Trên x86, “opt” chủ yếu = AVX2; số thật đã đo trên máy sinh viên: keygen ML-KEM-768 **30.985µs (ref) → 10.705µs (opt) = ×2.89**.

Nguồn: liboqs chính chủ <https://github.com/open-quantum-safe/liboqs>; phương pháp dùng speed tools là phương pháp OQS từng dùng trong hạ tầng đo của họ (<https://github.com/open-quantum-safe/profiling> — repo nay archive, ta cite làm tài liệu phương pháp, không dùng code).

Kiểm ngay: ~/pqc/src/liboqs/build-ref/tests/speed_kem ML-KEM-768 — banner phải in CPU extensions: SSE SSE2 (ref) còn cây opt in AVX2...; đó là bằng chứng hai cây khác đúng chỗ cần khác.

1.9 Bước 7 — fetch_pqclean.sh: nguồn duy nhất cho code-size từng thuật toán

```
bash scripts/fetch_pqclean.sh clean # ban C thuần, .a riêng từng thuật toán
bash scripts/fetch_pqclean.sh avx2 # (tùy chọn) ban SIMD để so kích thước
```

Làm gì: Clone PQClean (ghim theo *commit*), build các thư viện tĩnh libml-kem-768_clean.a, libml-dsa-65_clean.a... — mỗi thuật toán một file.

Tại sao: libcrypto.so/liboqs.so trộn *mọi* thuật toán trong một file — công cụ size không tách nổi “riêng ML-KEM chiếm bao nhiêu byte”. PQClean build mỗi scheme một .a → con số code-size *per-algorithm* (đề §9) chỉ có từ đây. So clean với avx2 còn trả lời “tốc độ đổi bằng bao nhiêu byte mã”. Ghim *commit* thay vì tag vì tag mới nhất của PQClean là round2/round3 cổ (tiền chuẩn hóa). Ghi nhận chính thức trong README dự án: PQClean sẽ archive read-only 07/2026, kế nhiệm là PQ Code Package — chi tiết này vào Literature của báo cáo.

Nguồn: <https://github.com/PQClean/PQClean>.

Kiểm ngay: script có rào *chắn chéo kiến trúc*: gọi aarch64 trên máy x86 sẽ bị chặn kèm hướng dẫn — rào này sinh ra từ một lần fail thật (assembly NTT viết tay không cross-compile được).

1.10 Bước 8 — build_oqsprovider.sh: đường tích hợp kế nhiệm

```
bash scripts/build_oqsprovider.sh ref # (tùy chọn ở mức độ chính)
```

Làm gì: Build oqs-provider 0.10.0 cắm vào OpenSSL của ta, theo đúng hai biến README chính chủ yêu cầu: OPENSSL_ROOT_DIR + liboqs_DIR.

Tại sao: Đề §7.2 nhắc “OpenSSL OQS fork” — nhưng fork 1.1.1 đó đã *ngừng phát triển*; README của chính nó (<https://github.com/open-quantum-safe/openssl>) trở người dùng sang **oqs-provider** (<https://github.com/open-quantum-safe/oqs-provider>). Ta làm theo lời khuyên chính chủ và ghi rõ trong báo cáo. Vai trò: bằng chứng tích hợp + mở thuật toán ngoài chuẩn; *số đo chính* vẫn từ ML-KEM/ML-DSA native của OpenSSL 3.5+ — một lớp ít hơn là một nguồn nhiễu ít hơn.

1.11 Bước 9 — gen_tls_certs.sh: chứng thư cho trục chữ ký

Lý thuyết — Certificate, trusted và untrusted — 5 phút nắm trọn. Trong TLS, *server* chia certificate (danh tính + khóa công khai, được *ai đó* ký); *client* kiểm “người ký có nằm trong danh sách tôi tin không”. Web thật: CA độc lập (Let’s Encrypt) ký → client tin sẵn → *trusted*. Đồ án: cert **tự ký** (Issuer = Subject = localhost — xem được bằng openssl x509 -noout -subject -issuer) → mặc định *untrusted* (client báo verify error 18). Cách xử có kiểm soát: copy file .cert.pem (phần *công khai* — được phép lan truyền) sang máy client rồi trao qua -CAfile → client verify ra 0 (ok). **Tuyệt đối không** copy .key.pem (bí mật của server; .gitignore cũng chặn). Chiều đi của cert: *server* → *client*; client trong TLS thường *không* có cert (mTLS hai chiều là chế độ đặc biệt, đồ án không dùng).

```
bash scripts/gen_tls_certs.sh
# mo rong ma tran: CERT_SET="mlsa44 mlsa87 rsa3072" FORCE=1 bash scripts/gen_tls_certs.sh
```

Làm gì: Sinh ba cặp khóa+cert tự ký vào ~/pqc/tls/: rsa2048 (baseline thực dụng), ecp256 (baseline elliptic), mlsa65 (hậu lượng từ mức 3).

Tại sao: Sinh khóa kiểu *hai bước* genkey rồi req -x509 -key: chỉ dùng cú pháp được tài liệu hóa chắc chắn cho thuật toán mới (genkey -algorithm ML-DSA-65 nằm nguyên văn trong manpage), né rủi ro cú pháp -newkey với tên alg lạ. Khóa nằm *ngoài* repo (~/pqc/tls) — khóa bí mật không bao giờ vào git.

Nguồn: tên thuật toán: https://docs.openssl.org/3.5/man7/EVP_PKEY-ML-DSA/.

1.12 Bước 10 — ĐO WP2: microbenchmark K-batch (số liệu xương sống)

```
sudo cpupower frequency-set -g performance # (1)
for i in 1 2 3 4 5; do # (2)
  BENCH_WARMUP=50 BENCH_ITERS=5000 make bench # (3)
  cp data/summary_micro_x86_64.csv \
    data/raw/x86_64/summary_batch$i.csv # (4)
done # (5)
```

Giải thích từng dòng:

1. Ghim CPU ở xung tối đa. Mặc định Linux dùng governor tiết kiệm điện — xung nhảy theo tải làm hai lần đo cùng một thuật toán ra hai số khác nhau. Để §7.4 yêu cầu đích danh (“fix CPU frequency”). Nguồn cơ chế: tài liệu cpufreq của kernel (link ở Bước 4).
2. Vòng K=5 *batch* độc lập — để §7.5: “run at least K batches (e.g., 5-10)”.
3. Một batch: make bench gọi run_micro.sh quét cả ma trận 12 thuật toán, mỗi cái M vòng. Hai biến env nâng vòng lên mức “lấy số thật” (mặc định 20/2000 chỉ là mức nháp). run_micro.sh ghi data/summary_micro_x86_64.csv dạng dài algo,metric,value — dạng dài để thêm metric không vỡ schema.
4. Cất bản batch vào data/raw/ với số thứ tự — 5 file này là *bằng chứng* K-batch và đầu vào cho phân tích sâu (bootstrap CI) ở tuần tổng hợp; analyze.py dùng bản summary mới nhất.
5. Hết vòng — trên x86 không cần nghỉ giữa batch (tản nhiệt PC đủ tốt; Pi thì khác, xem Phần II).

1.13 Bước 11 — ĐO WP5: bộ nhớ và kích thước mã

```
make memory # -> data/memory_x86_64.csv (algo, peak_rss_kb)
make codesize # -> data/codesize_x86_64.csv (file, text,data,bss,total)
size ~/pqc/src/PQClean/crypto_*/ml-*/clean/lib*.a \
| tee data/raw/x86_64/pqclean_clean_size.txt
```

Làm gì: make memory bọc từng thuật toán bằng /usr/bin/time -v và đọc dòng “Maximum resident set size” = **đỉnh RAM** tiến trình từng chạy. make codesize chạy size (binutils) lên libcrypto.so/liboqs.so lấy text/data/bss. Lệnh thứ ba đo *per-algorithm* qua các .a của PQClean.

Tại sao: /usr/bin/time là GNU time — để §7.4 gọi đích danh; chú ý *không phải* time builtin của shell (builtin không biết RSS). Ý nghĩa ba cột size: text = mã máy, data = hằng/bảng tra đã khởi tạo, bss = vùng zero-fill; tổng = “thuật toán nặng bao nhiêu trên đĩa/flash”. Caveat ghi thẳng vào script: số của libcrypto là số *trộn* mọi thuật toán — per-algorithm chỉ tin được từ PQClean.

Nguồn: man time(1), man size(1); để §7.4 nguyên văn “size utility, readelf -S... peak RSS via /usr/bin/time -v”.

1.14 Bước 12 — ĐO WP3: đo số ref/opt thành CSV

```
bash scripts/run_liboqs_speed.sh
# -> data/liboqs_speed_x86_64.csv (arch,variant,algo,op,iterations,mean_us,stdev_us,cycles_mean)
# -> data/raw/x86_64/liboqs_<variant>_<algo>.txt (output gốc làm bảng chung)
```

Làm gì: Chạy speed_kem/speed_sig của *cả hai cây* trên 6 thuật toán (3 KEM + 3 chữ ký), lưu output gốc, và parse bảng kết quả thành CSV.

Tại sao: Mặc định hai tool đó chỉ *in màn hình* — để §11.4 đòi “raw CSVs”. Parser awk được viết *ngược từ output thật* của lần chạy trên máy sinh viên (lọc theo cấu trúc: dòng có ≥ 7 cột phân tách | và cột 2 là số) — đã kiểm: 3 dòng CSV ra đúng từng chữ số (30.985/30.963/36.250). Script tự thích nghi: cây nào chưa build thì bỏ qua kèm thông báo — trên x86 chỉ có ref vẫn ra CSV hợp lệ.

Kiểm ngay: head -5 data/liboqs_speed_x86_64.csv thấy đủ cột; bảng speedup ref/opt sẽ tự mọc trong tables.md khi đủ *cả hai* variant cho ít nhất một thuật toán.

1.15 Bước 13 — ĐO WP4: TLS handshake, BA phương án cùng một thước đo

Lý thuyết — Handshake TLS 1.3 đo cái gì. Một kết nối HTTPS mở màn bằng *handshake*: client chào (kèm key-share của nhóm trao đổi khóa nó đoán server thích), server đáp (chọn nhóm, *chìa certificate*, ký để chứng minh danh tính), hai bên dẫn xuất khóa phiên. PQC chen vào đúng hai chỗ: nhóm trao đổi khóa (X25519MLKEM768 lai, MLKEM768 thuần) và chữ ký trong cert (ML-DSA-65). Tên nhóm lấy nguyên văn tài liệu https://docs.openssl.org/3.5/man3/SSL_CTX_set1_groups_list/ — trang này còn cho biết “DEFAULT list selects X25519MLKEM768 as one of the predicted keyshares” (OpenSSL 3.5+ *mặc định* đã ưu tiên lai hậu lượng tử).

Lý thuyết — Vì sao tắt resumption khi đo. TLS 1.3 có cơ chế nối lại phiên (session ticket/PSK): kết nối sau *bỏ qua* truyền certificate. Nếu để bật, từ kết nối thứ hai bạn đo một handshake *rút gọn* — số vô nghĩa cho việc so cert PQC (cert ML-DSA to gấp nhiều lần là điểm cần lộ ra!). Vì vậy mọi cấu hình đo đều tắt: -no_ticket/ssl_session_cache off + ssl_session_tickets off.

1.15.1 Phương án A — make tls (server = openssl s_server): cô lập mật mã

```
make tls # ~5-10 phút -> data/tls_handshake_x86_64.csv
```

Làm gì: bench_tls.sh dựng s_server với từng cert, rồi với từng group: (latency) lập TLS_ITERS handshake *tuần tự*, bấm giờ từng cái → median/mean/p95; (throughput) thả TLS_CONC client *song song* trong TLS_DUR giây → handshake/giây. “Client” là hàm one_handshake() bọc openssl s_client -groups ... -CAfile ... -verify_return_error.

Tại sao: s_server là server mỏng nhất có thể — gần như chỉ có libssl, nên số của nó *cô lập chi phí mật mã* khỏi mọi kiến trúc server. Cờ -verify_return_error bắt client *fail hẳn* nếu verify cert

rút → mọi handshake được đếm đều gồm verify thành công, đúng như client thật ngoài đời. **Vì sao không wrk:** wrk của distro link OpenSSL hệ thống (3.0, không ML-KEM) — chỉ được dùng làm đối chứng công cụ trên hàng X25519.

1.15.2 Phương án B — nginx: server sản xuất + trực đơn/đa luồng

B.1 — build_nginx.sh: dựng nginx “thuê” đúng OpenSSL.

```
| bash scripts/build_nginx.sh # x86 vài phút
```

Làm gì: Clone mirror chính chủ <https://github.com/nginx/nginx> tag release-1.30.2, rồi:

```
| auto/configure --prefix=$NGINX_PREFIX --with-http_ssl_module \
| --without-http_rewrite_module --without-http_gzip_module \
| --with-cc-opt="-I$SSL_PREFIX/include" --with-ld-opt="-L$SSL_LIBDIR"
```

Tại sao: từng cò: --with-http_ssl_module bật TLS (nginx mặc định tắt); --without-rewrite/gzip né kéo PCRE/zlib-dev (triết lý gói tối thiểu — hệ quả đã đụng thật: mất directive return, nên config trả *file tĩnh*); cặp cc-opt/ld-opt trở header+lib vào OpenSSL *của ta* = link **động** — tái dùng OpenSSL đã build (ARM khởi build lại ~1 giờ), một thư viện một nguồn sự thật; chạy lúc nào cũng qua setenv.sh (LD_LIBRARY_PATH) đúng quy ước no-rpath toàn repo. Cuối script tự kiểm nginx -V phải in built with OpenSSL 3.6 — chốt chặn bắt lỗi “thuê nhầm nhà” ngay tại bước build thay vì để nổ ở bước đo.

Kiểm ngay: đã build sống và kiểm trọn trong môi trường thử: nginx -V → built with OpenSSL 3.6.2; nginx -t *nhận* chuỗi group PQC và *từ chối* group bìa với lỗi lộ đúng hàm SSL_CTX_set1_curves_list(...) failed.

B.2 — nginx_bench.conf: giải thích TỪNG DÒNG.

```
| worker_processes __WORKERS__;  
| daemon off;  
| error_log logs/error.log warn;  
| pid logs/nginx.pid;  
| events { worker_connections 1024; }  
| http {  
|   access_log off;  
|   server {  
|     listen __LISTEN__:__PORT__ ssl;  
|     ssl_certificate __CRT__;  
|     ssl_certificate_key __KEY__;  
|     ssl_protocols TLSv1.3;  
|     ssl_ecdh_curve __GROUPS__;  
|     ssl_session_cache off;  
|     ssl_session_tickets off;  
|     location / { root html; }  
|   }  
| }
```

- worker_processes __WORKERS__ — script điền 1 hoặc auto: chính là trực “single-threaded and multi-threaded servers” để §7.4 đòi, thứ s_server không có.
- daemon off — nginx chạy *foreground*: script cầm trực tiếp PID để tắt sạch giữa các tổ hợp, không lẫn mò pid-file.
- error_log ... warn / pid logs/nginx.pid — đường ghi tương đối theo -p rundir: mỗi lần đo một thư mục tạm riêng, không vương vãi.
- worker_connections 1024 — trần kết nối mỗi worker; 1024 dư cho TLS_CONC mặc định 4.
- access_log off — tắt ghi log mỗi request: loại nhiễu I/O đĩa khỏi phép đo.

- `listen __LISTEN__:__PORT__ ssl — __LISTEN__ = 127.0.0.1` (đo loopback) hoặc `0.0.0.0` (đo qua LAN, Phần II/III).
- `ssl_certificate(__key)` — danh tính server; bên dưới nginx gọi `SSL_CTX_use_certificate/use_PrivateKey` (nguồn nginx `ngx_event_openssl.c:504/608`) — cùng hàm server tự viết của ta gọi.
- `ssl_protocols TLSv1.3` — khóa đúng một phiên bản: loại biến nhiều “thương lượng version”.
- `ssl_ecdh_curve __GROUPS__` — cửa ngõ PQC: tài liệu module (https://nginx.org/en/docs/http/ngx_http_ssl_module.html) định nghĩa là “list of curves supported by the server”; bên dưới là `SSL_CTX_set1_curves_list (:1799)` — tên cũ của `set1_groups_list`, một hàm hai tên. Xác nhận đường PQC: issue nginx #532; **bẫy đã tra**: ticket #2542 — directive này bị bỏ qua *lặng lẽ* ở server block không-default → mọi thứ nằm trong block default duy nhất.
- `ssl_session_cache off + ssl_session_tickets off` — ép mọi kết nối là full handshake (lý thuyết ở đầu Bước 13).
- `location / { root html; }` — trả file tĩnh `html/index.html` (“ok”) vì return thuộc module `rewrite` đã tắt; phản hồi vẫn tối giản.

B.3 — `bench_tls_nginx.sh`: chạy ma trận 18 ô.

```
bash scripts/bench_tls_nginx.sh
# -> data/tls_handshake_nginx-x86_64.csv (them cot server,workers)
```

Tại sao: Ba vòng lặp lồng nhau `workers (1,auto) × cert (3) × group (3) = 18` hàng; *phương pháp đo giữ y hệt* phương án A (cùng vòng latency, cùng bảo song song) — chú ý, để hai bộ số đối chứng nhau: A = chi phí mật mã thuần, B = thêm giá của một server thật + chiều đa lỗi; hiệu số chính là “giá của vỏ”. Tên file mang chữ `nginx-` để `analyze.py` tự nhặt và tự dán nhãn — không sửa một dòng code phân tích.

1.15.3 Phương án C — cặp client/server TỰ TRIỂN KHAI (đỉnh của §7.6)

Lý thuyết — “Tự triển khai” nghĩa là gì — ranh giới quan trọng. Tự sở hữu ba thứ: tầng TCP (socket/bind/listen/accept hoặc connect), tầng HTTP tối giản, và *luồng điều khiển + điểm đo*. Phần mật mã TLS vẫn thuê thư viện `libssl` — **đúng đắn và bắt buộc** (tự viết lại giao thức TLS vi phạm định luật “đừng tự chế crypto”; chính nginx cũng thuê `libssl`). Khác biệt với `s_client/s_server`: ta gọi `SSL_accept/SSL_connect` *trực tiếp trong C*, tự bấm giờ, tự chọn group — không giao cho một binary đóng kín rồi đọc text nó in.

```
make tlsmini tlsclient
# server (của số 1):
./build/tls_mini_server ~/pqc/tls/mlsa65.cert.pem ~/pqc/tls/mlsa65.key.pem 9443 \
| tee data/raw/x86_64/tlsmini_handshakes.csv
# client (của số 2):
./build/tls_timer_client 127.0.0.1 9443 X25519MLKEM768 50 ~/pqc/tls/mlsa65.cert.pem
```

Làm gì: Server (`src/tls_mini_server.c`, mẫu theo demo chính chủ <https://github.com/openssl/openssl/tree/master/demos/guide> — `tls-server-block.c`): TCP của ta → bàn giao fd qua đúng ba dòng `SSL_new → SSL_set_fd → SSL_accept → HTTP 200` tối giản, `Connection: close` để *mỗi kết nối = một full handshake = một đơn vị đo sạch*; kẹp `CLOCK_MONOTONIC_RAW` quanh `SSL_accept` và in mỗi kết nối một dòng `conn,tls,group,handshake_us` — cột group (qua `SSL_get_negotiated_group`) là *bằng chứng cứng* phiên đo thật sự chạy MLKEM768. **Client** (`src/tls_timer_client.c`, soi gương `s_timer.c` của Paquin-Stebila-Tamvada, mã tại <https://github.com/xvzcf/pq-tls-benchmark>, bài báo <https://eprint.iacr.org/2019/1447>): `BIO_s_connect → SSL_new → SSL_set_bio → SSL_connect` lặp N lần *trong cùng tiến trình*, bấm giờ từng `SSL_connect`, in CSV + median.

Tại sao: Client tự viết tồn tại vì `s_client` spawn một tiến trình mỗi handshake (chi phí fork/exec vài ms cộng đều) — đủ tốt cho *chênh lệch* giữa các group trên cùng máy, nhưng số *tuyệt đối* sạch cỡ paper thì cần đo trong-tiến-trình — đúng lý do giới học thuật viết `s_timer.c`. Hai phía đo hai nửa: client = handshake + round-trip + dựng kết nối; server = đúng đoạn `SSL_accept`; *hiệu hai* số tách chi phí đường truyền khỏi chi phí mật mã tại server. Cả hai đều fail-loudly với group lạ (`TLS_GROUPS`/đổi số group bịa → FATAL kèm lỗi gốc OpenSSL) — cùng cơ chế với nginx, vì cùng một hàm bên dưới.

Kiểm ngay: cặp này đã bắt tay sống: client median 1686µs, server ghi ~1500–1700µs cùng các kết nối (kết nối #1 lộ warm-up 7996µs); client ép TLS 1.2 bị server từ chối bằng alert protocol version — “TLS 1.3 only” có rằng.

1.16 Bước 14 — make analyze: từ CSV thành bảng và biểu đồ

```
| make analyze # -> analysis_out/tables.md + *.png
```

Làm gì: `analyze.py` quét *mọi* CSV trong `data/`, sinh: bảng median micro theo máy; bảng *Cross-platform ratio* (chỉ xuất hiện khi đủ hai máy — bắc cầu bằng tỉ lệ, định luật bốn-con-số); bảng+biểu đồ RSS, code size; bảng+biểu đồ TLS (grouped bar theo cert×group, riêng cho `s_server` và nginx); bảng *speedup ref→opt* của WP3.

Tại sao: Chỉ dùng csv chuẩn + matplotlib (pandas không bắt buộc — bớt một phụ thuộc); matplotlib văng vắn ra bảng (suy thoái có kiểm soát). Mọi schema đọc *đúng* schema thật của các script đo — và chính smoke-test đã bắt được một bug thật (tách tên kiến trúc hỏng vì `x86_64` chứa `_`) trước khi nó kịp ra trận.

Kiểm ngay: mở `analysis_out/tables.md`; thấy bảng “liboqs ref vs opt” với speedup ×2.6–2.9 (khi đủ hai cây) là chuỗi WP3 đã thông suốt từ binary tới biểu đồ.

1.17 Bước 15 — Docker (tái lập, tùy chọn trên x86)

```
| docker build -f docker/Dockerfile.x86_64 -t nt219-pqc . # tu GOC repo
| bash scripts/docker_buildx.sh # multiarch (can binfmt)
```

Tại sao: Dockerfile *tái dùng chính script của repo* (`build_openssl`, `make`, `verify_env`) — một nguồn sự thật; `verify_env.sh` cuối image khiến image *tự fail* khi hỏng. `buildx` dựng đa kiến trúc qua QEMU (<https://docs.docker.com/build/building/multi-platform/>) — vai trò là *bằng chứng đóng gói* §7.3; **định luật:** không bao giờ đo trong QEMU — số của trình giả lập, không phải của CPU.

1.18 Bước 16 — Tùy chọn x86 (mở rộng chiều dữ liệu) + nghiệm thu

```
| ~/pqc/openssl/bin/openssl speed rsa2048 ecdsap256 # doi chung cong cu cho WP2
| wrk -t4 -c16 -d10s https://127.0.0.1:8443/ # CHI hang X25519 (wrk link OpenSSL he thong)
| sudo tshark -i lo -f "tcp port 4433" -a duration:5 -q -z conv,tcp # bytes that tren day (9
|   network overhead)
| CERT_SET="mlsa44 mlsa87 rsa3072" FORCE=1 bash scripts/gen_tls_certs.sh
| TLS_CONC=8 TLS_DUR=15 make tls # tai nang hon
```

Nghiệm thu x86 (tick đủ mới sang Phần II): `verify_env` PASS · 5 file batch trong `data/raw/x86_64·memory/codesize/liboqs_speed/tls_handshake*` CSV có số · `analysis_out/tables.md` + PNG · `docs/openssl.commit` (và các `.commit` khác) tồn tại · **đã git push** — số chưa rời máy là số chưa tồn tại.

2 PHẦN II — THIẾT BỊ 2: ARM Raspberry Pi 4 (aarch64, Ubuntu)

2.1 ARM khác x86 ở đâu — lý thuyết bắt buộc trước khi thuê máy

Lý thuyết — Vì sao phải có máy ARM THẬT. RQ2 hỏi về hiệu năng *trên SBC* — và hiệu năng là thuộc tính của *phần cứng thật*: cache, pipeline, đơn vị SIMD. Giả lập (QEMU/buildx) cho binary chạy được nhưng *số đo là của trình giả lập*. Cross-compile trên PC cho ra binary ARM nhưng không cho ra con số nào. Kết luận khắc đá: **số ARM chỉ đến từ máy ARM**; cross/QEMU chỉ là bằng chứng năng lực đóng gói (§7.3).

Lý thuyết — NEON — “AVX2 của ARM”. x86 tăng tốc PQC bằng AVX2 (vector 256-bit); ARM dùng **NEON** (vector 128-bit, có mặt trên mọi Cortex-A). Phép nhân đa thức NTT trong ML-KEM/ML-DSA vector hóa rất tốt → kỳ vọng speedup ref→opt đáng kể, nhưng *khác* hệ số với x86 (vector hẹp hơn). So *tỉ lệ* speedup hai máy chính là câu trả lời RQ2.

Lý thuyết — Cortex-A72 KHÔNG có ARM Crypto Extensions. BCM2711 của Pi 4 thiếu phần cứng AES/SHA chuyên dụng. Hệ quả tốt cho thí nghiệm: cặp ref/opt trên Pi *sạch* — toàn bộ chênh lệch là NEON, không lẫn tăng tốc AES phần cứng. Ghi chi tiết này vào báo cáo khi giải thích số.

Lý thuyết — Nhiệt — kẻ phá đo thẩm lạnh. Pi 4 *throttle* (tự hạ xung) quanh ~80 °C. Batch đo dài làm nhiệt leo → xung tụt giữa chừng → batch sau chậm hơn batch trước *không phải vì thuật toán*. Đối sách: đọc nhiệt giữa các batch và nghỉ nguội — vòng lệnh ở Bước II.6.

Lý thuyết — Mạng IPv6-only / NAT64. Máy Pi hosted (ví dụ Mythic Beasts) thường chỉ có IPv6; truy cập GitHub (IPv4) đi qua NAT64/DNS64 của nhà cung cấp. Clone treo → kiểm cat /etc/resolv.conf và thử ping6 github.com. Đây cũng là lý do định luật bằng chứng nghiêm hơn ở Pi: máy thuê có thể bị thu hồi — **push trước khi rời máy**.

2.2 Pipeline ARM — toàn bộ lệnh, theo đúng thứ tự

II.1 — Lấy repo + cài gói (lý do từng gói y hệt Bước 1 Phần I):

```
git clone <URL-repo> && cd <ten-repo> && chmod +x scripts/*.sh
bash scripts/install_prereqs.sh
```

Tại sao: Cùng script chạy được cả hai kiến trúc vì apt tự chọn gói aarch64; linux-tools-\$(uname -r) trên Pi sẽ ra bản -raspi — đúng lý do script cài theo uname -r.

II.2 — Build OpenSSL (khác x86 một cò):

```
SKIP_TESTS=1 bash scripts/build_openssl.sh
# (một lần duy nhất, chạy qua dem de lay bang chung: bỏ SKIP_TESTS)
```

Tại sao: Pi build chậm (4 nhân A72, RAM 4GB): full test suite có thể thêm hàng giờ. Quy ước: bản dùng hằng ngày SKIP_TESTS=1; *một lần* qua đêm chạy đủ test làm bằng chứng đúng đắn trên chính kiến trúc này. JOBS từ versions.env = nproc = 4 — vừa sức RAM.

II.3 — Kích hoạt + cổng kiểm + harness:

```
source scripts/setenv.sh
bash scripts/verify_env.sh # PASS -> docs/env_report_aarch64.txt
make && make tlsmini tlsclient
```

Tại sao: verify_env trên Pi sinh *biên bản riêng của máy này* (_aarch64) — để §7.3 đòi tài liệu hóa *từng* nền tảng. bench_evpt tự nhận ARM lúc compile: bộ đếm chu kỳ chuyển từ rdtsc (x86) sang cntvct (ARM) — cùng một mã nguồn, hai kiến trúc.

II.4 — liboqs: TRỌNG TÂM của máy này — bắt buộc CẢ HAI cây:

```
bash scripts/build_liboqs.sh ref # NEON TAT tuong minh
bash scripts/build_liboqs.sh opt # NEON BAT (-mcpu=native)
```

Tại sao: Trên x86 cây opt là “nên có”; trên ARM nó là *lý do tồn tại của chuyến đo*: cặp ref/opt tại đây = thí nghiệm NEON = RQ2. Khối ARM trong script bật/tắt NEON *tường minh* để hai cây khác đúng một biến. Thiếu một cây → bảng “ref vs opt (aarch64)” sẽ *vắng mặt* trong tables.md — checklist cuối phần bắt đúng lỗ hổng này.

II.5 — PQClean hai trục + chứng thư:

```
bash scripts/fetch_pqclean.sh clean
bash scripts/fetch_pqclean.sh aarch64 # truc NEON thu hai (per-algo .a)
size ~/pqc/src/PQClean/crypto_kem/ml-kem-768/{clean,aarch64}/lib*.a \
| tee data/raw/aarch64/pqclean_neon_size.txt
bash scripts/gen_tls_certs.sh # cert sinh LAI tren may nay
```

Tại sao: aarch64 của PQClean chỉ build được *trên máy ARM* (assembly NTT viết tay — rào chắn trong script chặn từ x86 là vì vậy); cặp clean/aarch64 trả lời “NEON đổi tốc độ bằng bao nhiêu byte mã”. Chứng thư sinh lại tại chỗ: khóa là tài sản theo máy, không copy chéo.

II.6 — ĐO WP2 với vòng nghỉ-nguội, giải thích TỪNG DÒNG:

```
sudo cpupower frequency-set -g performance # (1)
for i in 1 2 3 4 5; do # (2)
  BENCH_ITERS=1500 make bench # (3)
  cp data/summary_micro_aarch64.csv \
    data/raw/aarch64/summary_batch$i.csv # (4)
  cat /sys/class/thermal/thermal_zone0/temp # (5)
  sleep 120 # (6)
done
```

1. Ghim xung — Pi mặc định governor ondemand; nếu máy thiếu cpupower: `echo performance | sudo tee /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor`.
2. Vặn K=5 batch như x86 — phương pháp đồng nhất hai máy thì tỉ lệ mới so được.
3. Vòng *thấp hơn* x86 (1500 vs 5000): batch ngắn → nhiệt leo ít → tránh chạm ngưỡng throttle giữa batch.
4. Cất bằng chứng batch, y hệt x86.
5. **Đọc nhiệt** — giá trị phần nghìn độ C (70000 = 70.0°C); ngưỡng tự đặt: *dưới 70000 mới chạy batch* kế. Nguồn cơ chế: sysfs thermal của kernel.
6. Nghỉ 2 phút cho tản nhiệt thụ động của Pi hạ nhiệt — đổi thời gian lấy độ tin cậy.

II.7 — WP5 + WP3 + WP4 (lệnh y hệt x86, khác chỗ nào ghi chú đó):

```
make memory && make codesize
bash scripts/run_liboqs_speed.sh # can DU ref+opt -> 12 thuật toán x 2 variant
bash scripts/build_nginx.sh # Pi ~5-10 phút (KHÔNG rebuild OpenSSL - link dong)
make tls # phuong an A
bash scripts/bench_tls_nginx.sh # phuong an B (TLS_ITERS=30 neu muon ngan)
./build/tls_mini_server ~/pqc/tls/mldsa65.cert.pem ~/pqc/tls/mldsa65.key.pem 9443 \
| tee data/raw/aarch64/tlsmini_handshakes.csv &
./build/tls_timer_client 127.0.0.1 9443 X25519MLKEM768 50 ~/pqc/tls/mldsa65.cert.pem
make analyze
```

Tại sao: Mọi script tự nhận `uname -m` → file ra mang đuôi aarch64, không đề số x86. analyze chạy trên máy nào cũng được — nhưng để bảng *Cross-platform ratio* xuất hiện, thư mục data/ phải chứa CSV của *cả hai* máy (kéo về một chỗ qua git là cách chuẩn).

II.8 — PUSH TRƯỚC KHI RỜI MÁY (định luật bằng chứng, phiên bản nghiêm):

```
| git add data analysis_out docs && git commit -m "aarch64 measurements" && git push
```

Tại sao: Máy thuê có thể bị thu hồi/reimage bất kỳ lúc nào — số chưa push là số sẽ biến mất. Push xong mới được phép tắt phiên SSH.

2.3 Cross-compile từ x86 — bằng chứng năng lực (KHÔNG phải nguồn số)

```
| sudo apt install gcc-aarch64-linux-gnu g++-aarch64-linux-gnu # tren PC x86
| bash scripts/cross_build_liboqs.sh
| file ~/pqc/liboqs-aarch64-cross/lib/liboqs.so* # phai in: ARM aarch64
```

Làm gì: Build liboqs *cho* ARM ngay trên PC, dùng cmake/aarch64-toolchain.cmake. Giải thích từng dòng toolchain:

- CMAKE_SYSTEM_NAME Linux + CMAKE_SYSTEM_PROCESSOR aarch64 — khai “đích là Linux/ARM64”: CMake chuyển sang chế độ cross.
- CMAKE_C(XX)_COMPILER aarch64-linux-gnu-gcc/g++ — dùng compiler chéo của Ubuntu.
- Bộ tứ FIND_ROOT_PATH_MODE: PROGRAM=NEVER (công cụ chạy lúc build vẫn là đồ host), LIBRARY/INCLUDE/PACKAGE=ONLY (thư viện/header *chỉ* tìm trong sysroot đích — cấm link nhầm lib x86). Khuôn theo cmake-toolchains(7) và soi gương file mẫu liboqs ship sẵn (.CMake/toolchain_rasppi.cmake — bản của họ là armhf/gcc-8 32-bit, bản ta là aarch64 hiện đại).
- Script truyền thêm -DOQS_USE_OPENSSL=OFF — host không có OpenSSL bản ARM để link.

Tại sao: Vai trò duy nhất: để §7.3 đòi “separate cross-compile scripts (aarch64-linux-gnu toolchain)” — đây là bằng chứng *biết đóng gói cho ARM*; binary sinh ra là generic ARM, **không bao giờ** là nguồn số đo. Đã kiểm sống: cross-configure nhận GNU 13.3.0 aarch64, “Configuring done”, 0 cảnh báo.

2.4 Tùy chọn ARM

```
| USE_ARM_PMU=1 FORCE=1 bash scripts/build_liboqs.sh opt # CHI sau khi nạp module pqax
| FORCE=1 bash scripts/build_openssl.sh # full test qua dem (1 lan)
```

Tại sao: PMU (Performance Monitoring Unit) cho liboqs đọc chu kỳ *chính xác mức phản ứng*, nhưng cần nạp kernel module (dự án pqax) và *nhiều cloud không expose PMU* — vì vậy là tùy chọn; thiếu nó speed_* vẫn báo thời gian tường (đủ cho RQ2).

Nghiệm thu ARM: 5 batch + log nhiệt · liboqs_speed_aarch64.csv đủ cả *ref+opt* · memory/codesize CSV · pqclean clean+aarch64 size · tls_handshake_aarch64.csv (+ bản nginx) · tlsmini CSV · env_report_aarch64.txt · **đã push**.

3 PHẦN III — Sau khi có số ở cả hai máy: tổng hợp, báo cáo, demo

```
git pull && make analyze # gop CSV hai máy -> bảng Cross-platform ratio xuất hiện
cd docs/report && xelatex main.tex && xelatex main.tex # báo cáo (maybệfig tu hiện hình)
bash scripts/demo.sh # quay màn hình ~2 phút (Deliverable 5)
```

Tại sao: Đọc bảng ratio đúng cách: *không* kết luận “ARM chậm hơn x86 N lần là tính chất của PQC” (đó là tính chất của hai CPU); câu trả lời RQ2 nằm ở so sánh **speedup ref→opt** của hai máy — nếu NEON kéo được hệ số tương đương AVX2, giả thuyết “tối ưu phần mềm đưa PQC về mức khả thi trên SBC” đứng vững. **Đo qua LAN** (mở rộng RQ3): máy server bật BIND_ALL=1 (server tự viết) hoặc __LISTEN__=0.0.0.0 (nginx) + mở firewall sudo ufw allow 8443/tcp; máy client copy *cert public* sang (scp ...cert.pem) rồi tls_timer_client <IP-server> 8443 <group> 50 cert.pem — chênh lệch với số loopback = chi phí mạng thật, nơi ClientHello ~1KB của ML-KEM có thể lộ chuyện phân mảnh gói.

A Phụ lục A — Bản đồ 51 file: mỗi file một dòng công dụng

File	Công dụng
Makefile	công tắc tổng 8 target; tự dò lib/lib64; fail-to-be-loud khi thiếu OpenSSL
README.md	cửa chính: quickstart 2 máy, bảng chọn phương án, bảng biến env, nghiệm thu
.gitignore	chặn build/, nguồn bên thứ ba, *.key(.pem); <i>không</i> chặn data/ và analysis_out/ (deliverable)
src/bench_evp.cpp	harness WP2: EVP, 12 thuật toán, CLOCK_MONOTONIC_RAW + cycles, 4 biến env
src/tls_mini_server.c	server HTTPS tự viết: TLS1.3-only, đo SSL_accept phía server, in group, BIND_ALL
src/tls_timer_client.c	client đo tự viết: lập SSL_connect trong-tiến-trình, CSV + median, ép group
scripts/versions.env	ghim mọi tag + prefix (một nguồn sự thật)
scripts/setenv.sh	kích hoạt môi trường (source!); no-rpath + LD_LIBRARY_PATH
scripts/install_prereqs.sh	ba đợt gói ánh xạ \$8.2/\$7.4/\$16; cố ý loại libssl-dev
scripts/build_openssl.sh	nền móng 3.6.2; install_ssldirs; test suite; openssl.commit
scripts/verify_env.sh	cổng chặn + biên bản env_report_<arch>
scripts/build_liboqs.sh	hai cây ref/opt — thí nghiệm RQ2; USE_ARM_PMU
scripts/fetch_pqclean.sh	.a per-algorithm; ghim commit; rào chắn chéo kiến trúc
scripts/build_oqsprovider.sh	provider kế nhiệm fork; OPENSSL_ROOT_DIR+liboqs_DIR
scripts/cross_build_liboqs.sh	cross aarch64 trên x86 — bằng chúng \$7.3
cmake/aarch64-toolchain.cmake	toolchain cross theo cmake-toolchains(7), soi gương mẫu liboqs
scripts/docker_buildx.sh	+ đóng hộp tái lập; image tự-fail qua verify_env; QEMU
docker/Dockerfile.x86_64	không đo
scripts/run_micro.sh	quét 12 thuật toán → summary_micro (long format) + raw
scripts/measure_memory.sh	peak RSS qua GNU time -v
scripts/measure_codesize.sh	text/data/bss qua size; CODESIZE_LIBS
scripts/run_liboqs_speed.sh	WP3 runner: 2 cây × 6 thuật toán → CSV + raw evidence

scripts/gen_tls_certs.sh	3 cert tự ký (CERT_SET mở rộng); genpkey 2 bước; khóa ngoài repo
scripts/bench_tls.sh	WP4-A: s_server; latency tuần tự + bảo song song; NA khi fail
scripts/build_nginx.sh	WP4-B: nginx 1.30.2 link động OpenSSL ta; chốt chặn nginx -V
scripts/nginx_bench.conf	template đo (tùng dòng ở Bước 13); placeholder __LISTEN__ cho LAN
scripts/bench_tls_nginx.sh	ma trận 18 ô (workers×cert×group) → CSV nhãn nginx
scripts/analyze.py	gom mọi CSV → tables.md + PNG; ratio 2 máy; speedup WP3
scripts/demo.sh	Deliverable 5: 3 màn quay 2 phút
scripts/init_repo.sh/.ps1	tiện ích khởi tạo repo hai hệ điều hành
docs/KICH_BAN_TRIEN_KHAI.md	kịch bản đo đầy đủ 2 máy + checklist
docs/THIET_KE_CHI_TIET.md	hồ sơ thiết kế: mọi quyết định + nhật ký bằng chứng 10 mục
docs/report/main.tex	khung báo cáo XeLaTeX, 9 nguồn kiểm sống, maybefig
docs/WP1_*	tra cứu nhanh môi trường + bản in PDF
data/, analysis_out/	số + bảng/hình — được commit (deliverable §11.4)

B Phụ lục B — Toàn bộ biến môi trường (không hardcode)

Cú pháp: đặt trước lệnh — TLS_ITERS=100 TLS_CONC=8 make tls; cả make bench BENCH_ITERS=5000 cũng tới đích (đã test sống). WP2: BENCH_ITERS 2000 · BENCH_KEYGEN_ITERS 50 (RSA/EC) / 200 (PQC — keygen nhanh nên cần nhiều mẫu) · BENCH_WARMUP 20 · BENCH_CSV đường dẫn mẫu thô. WP4: TLS_ITERS 50 · TLS_WARMUP 5 · TLS_CONC 4 · TLS_DUR 10 · PORT 4433/8443 · CERTS · GROUP_LIST · (nginx) WORKERS_LIST "1 auto". Cert: CERT_SET · FORCE. WP3: KEMS · SIGS. WP5: CODESIZE_LIBS. Server tự viết: TLS_GROUPS (fail-loudly) · BIND_ALL=1 (LAN). Build: FORCE=1 (openssl/liboqs/nginx/certs) · SKIP_TESTS=1 · USE_ARM_PMU=1 · JOBS. *Cố ý đúng yên*: ma trận 12 thuật toán trong run_micro/measure_memory — đó là hằng số thí nghiệm §7.1, đổi nó là đổi thiết kế nghiên cứu; đo lẽ đã có ./build/bench_evpm lkem 768.

C Phụ lục C — FAQ “tại sao” cho người mới (12 câu hay gặp nhất)

1. *Sao không apt install nginx cho nhanh?* — nginx của apt link OpenSSL 3.0 hệ thống: không có ML-KEM; sửa config kiểu gì cũng chết ở SSL_CTX_set1_curves_list failed vì lỗi nằm ở thư viện bên dưới.
2. *Sao phải hai máy thật?* — hiệu năng là thuộc tính phần cứng; RQ1/RQ2 hỏi về hai lớp phần cứng khác nhau.
3. *Sao không so thẳng số x86 với số ARM?* — định luật bốn-con-số: khác máy chỉ so tỉ lệ.
4. *Median hơn gì mean?* — miễn nhiễm vài vòng đột biến (nhiều hệ điều hành); mean bị một outlier kéo lệch.
5. *Warm-up để làm gì?* — vòng đầu cache lạnh: bằng chứng sống conn#1 7996µs vs #2-3 ~3800µs.
6. *Sao phải tắt session resumption khi đo TLS?* — không tắt thì từ kết nối 2 là handshake rút gọn, mất phần truyền cert — đúng phần PQC khác biệt nhất.

7. *Copy cert sang máy client là trusted hay untrusted?* — là biến untrusted thành trusted có kiểm soát: bạn tự đóng vai CA cho phòng lab; chỉ copy .cert.pem (công khai), không bao giờ .key.pem.
8. *Client có gửi cert cho server không?* — Không (TLS một chiều như mọi web); chiều cert là server→client; hai chiều là mTLS — chế độ đặc biệt, ngoài phạm vi.
9. *“Tự triển khai” mà vẫn #include openssl — có gian không?* — Không: tự sở hữu TCP + HTTP + luồng điều khiển + điểm đo; phần giao thức TLS thuê libssl là *bắt buộc đúng đắn* (đừng tự chế crypto) — nginx cũng vậy.
10. *Ba server A/B/C có thừa không?* — ba tầng cô lập: mật mã thuần → +vỏ server thật/đa lỗi → +điểm đo phía server; ba số kể một chuyện thì kết luận vững gấp ba.
11. *Sao mọi thứ ghim phiên bản mà không lấy bản mới nhất?* — “mới nhất” hôm nay ≠ ngày mai → không tái lập; tag bất biến; đổi phiên bản là quyết định có hồ sơ (sửa versions.env).
12. *Số đo xong để trên máy được không?* — Không, nhất là máy thuê: chưa push = chưa tồn tại.

Nguồn tổng (mọi link đã kiểm trong quá trình xây repo)

- OpenSSL docs 3.5: EVP_PKEY-ML-KEM / EVP_PKEY-ML-DSS / SSL_CTX_set1_groups_list — <https://docs.openssl.org/3.5/>; demos chính chủ (sslecho, guide/tls-server-block.c) — <https://github.com/openssl/openssl/tree/master/demos>
- Chuẩn: FIPS 203 / FIPS 204 / SP 800-57 — <https://csrc.nist.gov/>
- liboqs <https://github.com/open-quantum-safe/liboqs> · oqs-provider <https://github.com/open-quantum-safe/oqs-provider> · oqs-demos <https://github.com/open-quantum-safe/oqs-demos> · profiling (archived, phương pháp) <https://github.com/open-quantum-safe/profiling> · PQClean <https://github.com/PQClean/PQClean> · pqm4 <https://github.com/mupq/pqm4>
- nginx: mirror nguồn <https://github.com/nginx/nginx> · module SSL https://nginx.org/en/docs/http/nginx_ssl_module.html · ticket #2542 (bẫy default-server)
- Học thuật: Paquin-Stebila-Tamvada, *Benchmarking post-quantum cryptography in TLS* — <https://eprint.iacr.org/2019/1447>; mã <https://github.com/xvzcf/pq-tls-benchmark>
- Kernel cpufreq/thermal sysfs — <https://www.kernel.org/doc/html/latest/admin-guide/pm/cpufreq.html>; Docker multi-platform — <https://docs.docker.com/build/building/multi-platform/>